

Note about mult2: does not need a stack frame
since all its data stored in registers
and it is not a caller.

Example – Steps 5 and 6

```
0000000000400540 <mult_store>:  
1. 400540: push    %rbx          # Save %rbx  
2. 400541: mov     %rdx,%rbx        # Save dest  
3. 400544: callq   400550 <mult2>    # mult2(x,y)  
   400549: mov     %rax, (%rbx)        # Save at dest  
   40054c: pop     %rbx                # Restore %rbx  
   40054d: retq                      # Return
```

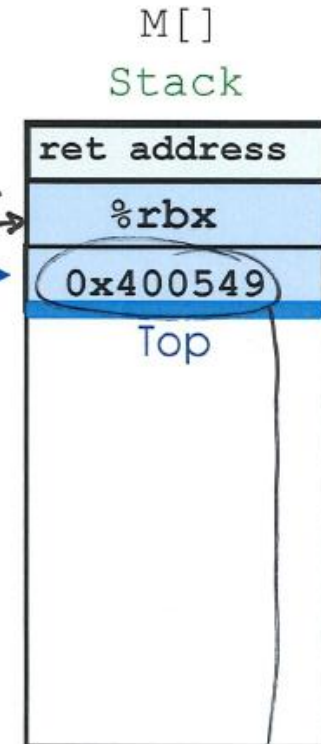
```
0000000000400550 <mult2>:  
4. 400550: mov     %rdi,%rax          # a  
5. 400553: imul    %rsi,%rax          # a * b  
6. 400557: retq                      # Return
```

%rdi **x -> a**
%rsi **y -> b**
%rdx **dest**

%rbx **dest**
%rax **a * b**
(5.2)

%rsp **0x110**
%rip **0x400553**
PC
" 557 (5.1)
" 558 (6.1)
" 549 (6.2)

(6.3)
%rsp + 8
%rsp



Example – Steps 7, 8 and 9

```

0000000000400540 <mult_store>:
1. 400540: push    %rbx          # Save %rbx
2. 400541: mov     %rdx,%rbx        # Save dest
3. 400544: callq   400550 <mult2>    # mult2(x,y)
7. 400549: mov     %rax, (%rbx)      # Save at dest
8. 40054c: pop     %rbx             # Restore %rbx
9. 40054d: retq                    # Return
    
```

```

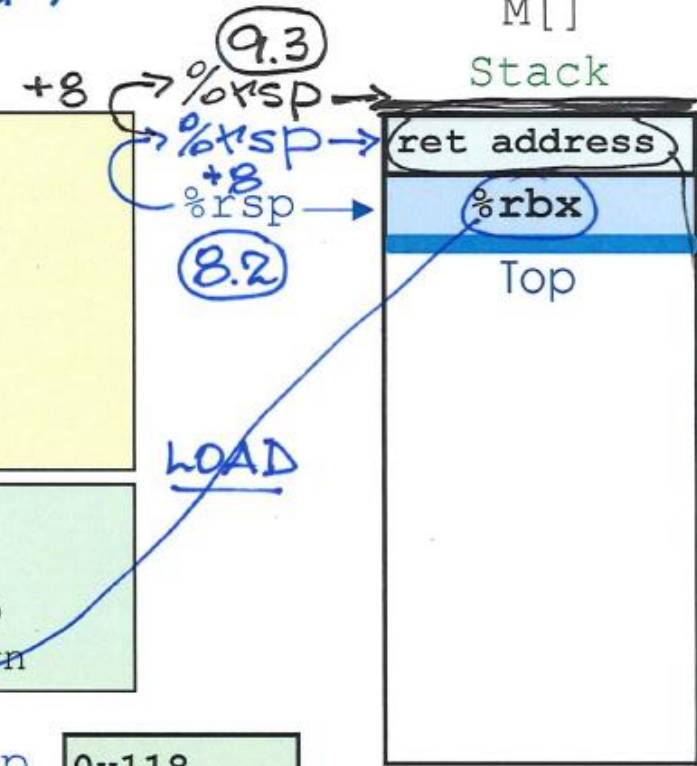
0000000000400550 <mult2>:
4. 400550: mov     %rdi,%rax        # a
5. 400553: imul    %rsi,%rax        # a * b
6. 400557: retq                    # Return
    
```

%rdi x
 %rsi y
 %rdx dest

value of %rbx
 %rbx dest 8.3 %rsp
 %rax a * b %rip

7.2 M[dest]

but we do not know this memory address



LOAD

" ~~54C~~ 7.1
 " ~~54D~~ 8.1
 " 54E 9.1
 ret.add. 9.2

of caller which called mult_store